SPEAKER HUB · speaker-hub.com

Site Review & Security Report

Automated maintenance run · July 6, 2026

1	1	3	78/78
SSRF vuln fixed	Dependency CVE patched	Legal gaps closed	API routes audited clean

Summary. I reviewed the whole codebase — all 78 API routes, the Supabase database (live advisor lints + RLS policies), the legal pages, SEO/metadata, and dependencies. The app is **already well-secured**: auth checks, ownership filters, encrypted secrets-at-rest, HMAC admin links, Stripe signature verification, and XSS escaping are all in place. I fixed **one real server-side vulnerability**, **patched a Next.js CVE**, **closed three legal-disclosure gaps** the app had outgrown, improved SEO, and prepared a database-hardening migration for you to review. The production build passes: compiled, typechecked, and all 74 pages generated. A short list of items only you can action is at the end.

1 · Changes made in this run

AREA	FILE	CHANGE	STATUS
Security	`lib/brand/extract.ts`	Blocked SSRF on the brand-extract URL fetch (scheme + private-host guard + post-redirect recheck)	FIXED
Security	`package.json`	Next.js `14.2.29 → 14.2.35` (patches a known CVE)	FIXED
Legal	`app/privacy/page.tsx`	Added Google/Gmail/Calendar, Upload-Post, and booking-inquiry disclosures; bumped date	UPDATED
Legal	`app/terms/page.tsx`	Added connected-Google-account clause; bumped date	UPDATED
SEO	`app/sitemap.ts`	Added public `/about` and `/donate` pages to the sitemap	FIXED
Database	`supabase/security_hardening.sql`	New reviewable migration for the Supabase advisor findings (not auto-applied)	REVIEW

2 · Security findings

HIGH SSRF — brand extractor fetched arbitrary URLs FIXED

`lib/brand/extract.ts` · entered from `POST /api/brand/extract`

The "extract my brand from my website" tool fetched the user-supplied URL with no protection. `normalizeUrl()` only checked that the hostname contained a dot; the existing `isBlockedHost()` guard was applied *only* to scraped stylesheet URLs, never to the primary fetch. A signed-in user could POST `http://169.254.169.254/…` (cloud metadata), `http://127.0.0.1:port`, or any internal address and make the server fetch it — and part of the response (the page description) is reflected back, so it is not even blind.

Fix: the primary fetch now rejects non-`http(s)` schemes and any private/loopback/link-local host *before* connecting, and re-checks the final host after redirects (defends against a public URL that 302-redirects inward). This mirrors the guard the sibling `/api/seo` route already had.

HIGH Dependency CVE — Next.js 14.2.29 FIXED

The pinned `next@14.2.29` is flagged by npm as carrying a known security vulnerability (Next.js security advisory, 2025-12-11). Bumped to `14.2.35` — a patch release within the same minor line, so no breaking changes. Build re-verified green afterward.

MEDIUM **Supabase advisor lints** MIGRATION PROVIDED

Ran the live Supabase security & performance advisors. Findings and remediations:

- **Anonymous-access policies** (WARN, ~35 tables): RLS policies are granted to the `public` role, which includes `anon`. Safe in practice — `auth.uid()` is NULL for anonymous requests so no row ever matches — but best practice is to scope them `to authenticated`. The migration demonstrates the exact pattern.
- **RLS re-evaluated per row** (perf, 3 policies): the DELETE policies on `feedback_points` / `engagement_timeline` and `own user_memories` still call `auth.uid()` per row. The migration wraps them in `(select auth.uid())` so Postgres evaluates once.
- **Leaked-password protection disabled** (WARN): a dashboard toggle, not SQL — see action items.
- `access_requests` **RLS with no policy** (INFO): intentional and safe — the table denies all anon/authenticated access; only the service-role API route writes to it. Documented so it isn't mistaken for a gap.

I did **not** apply these to the live database automatically — schema changes to production are yours to review and run: `supabase/security_hardening.sql`.

AUDITED — CLEAN **Authorization, IDOR, secret exposure**

A full pass over all 78 API routes found **no** authorization or data-access bugs:

- Every non-public route authenticates; the public set is exactly the expected one (Stripe webhook, public inquiry, request-access, signup/resend, donate).
- Every `[id]` route scopes its query by the owner's `user_id` (no IDOR). Service-role queries never trust a client-supplied id.
- BYOK API keys and OAuth tokens are never returned in plaintext — only masked 4-char hints or `{connected:true}` booleans. Tokens are AES-256-GCM encrypted at rest.
- Admin endpoints gate on the admin email; the emailed one-click approve/deny links use HMAC-signed, expiring, constant-time-verified tokens that fail closed.

LOW **Residual items noted (not changed)**

- **Redirect / DNS-rebinding SSRF residual** — both the brand and SEO fetchers follow redirects and validate hostnames as strings. A fully robust fix (resolve DNS, block private IPs at connect time, manual redirect handling) is a larger change; the high-severity hole is closed and the residual is low-risk.
- **CSV formula injection** in the analysis export (`/api/analyses/[id]/export`): cells beginning with `= + - @` aren't prefix-guarded. Fields are mostly AI-generated; low exposure. Optional one-line hardening.

3 · Legal pages — kept in sync (required by CLAUDE.md)

The privacy policy and terms had fallen behind the app's actual data handling. Per the project's legal-maintenance rule I updated them:

- **Connected Google account (Gmail + Calendar).** The AI Assistant requests Gmail read/compose/send and Calendar read scopes and can draft/send email on the user's behalf — this was **entirely undisclosed**. Added a dedicated privacy section (including a Google API Services User Data Policy / Limited-Use statement) and a terms clause.
- **Upload-Post subprocessor.** Clips and connected-account credentials are routed through `upload-post.com` for publishing. Added it to the subprocessor list.
- **Booking inquiries from event organizers.** The public one-sheet collects personal data about non-users (organizers). Added a clause explaining this collection and that the speaker, not Speaker Hub, is the recipient.

Both `updated` dates bumped to July 6, 2026. These are a baseline, not legal advice — see action items.

4 · Verification

- `npx tsc --noEmit` — passes (0 errors).
- `npm run build` — compiled successfully, linting + type validity checked, all 74 static pages generated (exit 0 with env present).
- SSRF fix reviewed against the code path; mirrors the already-correct `/api/seo` guard.

5 · Action items for Mick

#	ACTION	WHY
1	Review & run `supabase/security_hardening.sql` in the Supabase SQL editor	Clears the advisor RLS/perf lints; safe & idempotent
2	Enable **Leaked Password Protection** (Dashboard → Authentication → Password)	Rejects breached passwords; can't be set via SQL
3	Have **counsel review** the privacy/terms edits; the Gmail scopes also require Google OAuth app verification + Limited-Use compliance	Restricted Google scopes are effectively mandatory to disclose
4	Confirm the live plan prices ($0/$19/$49) — `lib/subscription/plans.ts` still comments "Prices are placeholders"	Avoid shipping placeholder numbers as real pricing
5	Ensure `NEXT_PUBLIC_APP_URL` is set in production	The Google OAuth `redirectUri` falls back to `localhost:3002` if unset
6	Optional: deploy so the new Next patch, sitemap, and legal pages go live	Changes are on the feature branch / PR